ASP.NET Web API

Tag 4 – Fehlerbehandlung, Sicherheit und Betrieb

4-Tage-Kurs | Alexander Fischer, MSc

Wiederholung: Was wisst ihr noch?

1. Welche zwei Befehle mit dem dotnet-ef Tool bringen eine Entity-Änderung in die Datenbank?
   - `dotnet ef migrations add <Name>`, dann `dotnet ef database update`
2. Warum ist der QuoteService jetzt Scoped?
   - Er nutzt den DbContext, und der ist Scoped (ein Singleton darf kein Scoped halten)
3. Was ist falsch an `_logger.LogInformation($"Buch {id}")`?
   - Interpolation erzeugt fertigen Text: Platzhalter `"Buch {BookId}", id` nutzen, sonst kein Filtern nach Feldern
4. Was bedeutet Arrange, Act, Assert?
   - Vorbereiten, Methode aufrufen, Ergebnis prüfen
5. Welchen Vorteil hat eine `.http`-Datei gegenüber Postman?
   - Liegt im Projekt und in Git, läuft direkt in der IDE

Exceptionhandling

Was passiert bei einer unbehandelten Exception?

Umgebung	Antwort
Development	Fehlerseite mit Stacktrace
Production	leere `500`-Antwort

- Der Client erfährt nicht, was los ist
- Stacktraces in Produktion wären ein **Sicherheitsrisiko**
- Ziel: einheitliches, sauberes Fehlerformat

try/catch vs. globale Behandlung

	try/catch in jeder Aktion	Globaler Handler
Code	aufgebläht	eine Stelle
Vergessen	leicht	unmöglich
Format	uneinheitlich	einheitlich
Logging	verstreut	zentral

> try/catch nur, wo ihr **sinnvoll reagieren** könnt (Retry, Fallback).

Exceptions sind keine Steuerlogik

Situation	Behandlung
Zitat nicht gefunden	Rückgabewert → `404`
Maximum erreicht	Rückgabewert → `409`
Ungültige Eingabe	Validierung → `400`
Datenbank weg	Exception → globaler Handler → `500`
Bug im Code	Exception → globaler Handler → `500`

IExceptionHandler

```
public class GlobalExceptionHandler : IExceptionHandler
{
    private readonly ILogger<GlobalExceptionHandler> _logger;
    public GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger)
        => _logger = logger;

    public async ValueTask<bool> TryHandleAsync(
        HttpContext ctx, Exception ex, CancellationToken ct)
    {
        _logger.LogError(ex, "Unbehandelter Fehler bei {Path}", ctx.Request.Path);
        // Status setzen, ProblemDetails schreiben
        return true;   // behandelt
    }
}
```

Den Handler registrieren

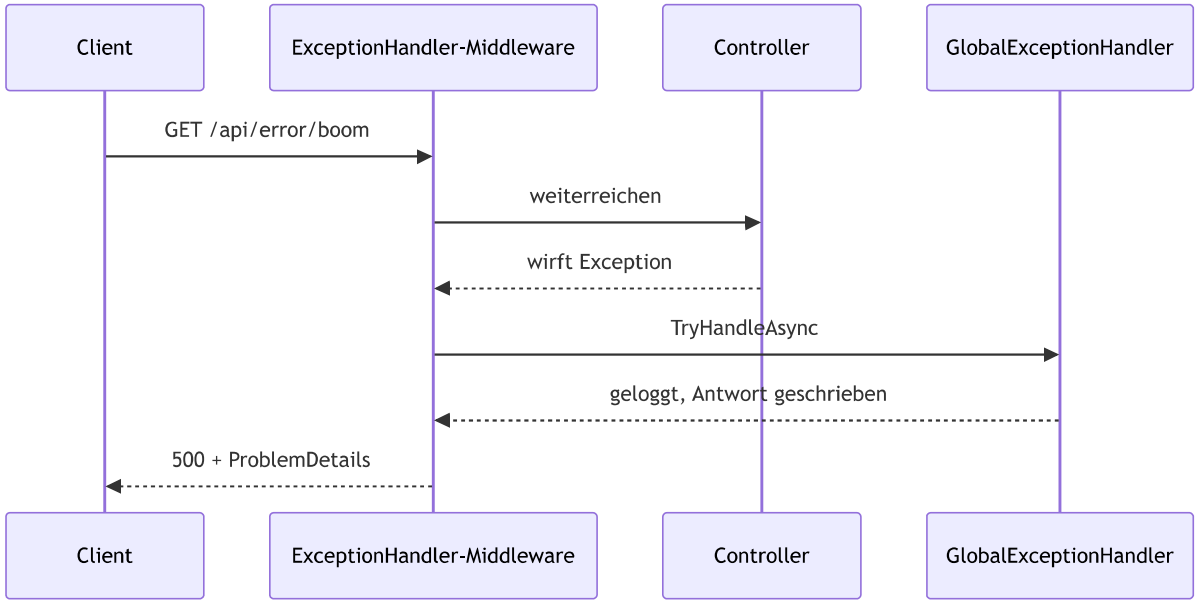
```
// Dienste
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();

var app = builder.Build();

// Pipeline – möglichst weit vorne
app.UseExceptionHandler();
```

Mehrere Handler möglich – sie werden der Reihe nach probiert, bis einer `true` liefert.

Ablauf bei einer Exception



Übung 14 – Globale Fehlerbehandlung

15 Minuten

1. `GET /api/error/boom` wirft eine Exception
2. Globaler Handler loggt und liefert ProblemDetails mit Status 500
3. Handler registrieren, `UseExceptionHandler()` in die Pipeline

Bonus: `KeyNotFoundException` wird zu `404` (Endpoint `/api/error/missing`)

Lösung Übung 14 (1/2)

```
public async ValueTask<bool> TryHandleAsync(
    HttpContext ctx, Exception ex, CancellationToken ct)
{
    _logger.LogError(ex, "Unbehandelter Fehler bei {Path}", ctx.Request.Path);

    var status = ex is KeyNotFoundException ? 404 : 500;
    ctx.Response.StatusCode = status;
    await ctx.Response.WriteAsJsonAsync(new ProblemDetails
    {
        Status = status,
        Title = status == 404 ? "Nicht gefunden" : "Interner Fehler",
        Detail = "Bitte die Anfrage prüfen oder später erneut versuchen."
    }, ct);
    return true;
}
```

Lösung Übung 14 (2/2)

```
[ApiController]
[Route("api/[controller]")]
public class ErrorController : ControllerBase
{
    [HttpGet("boom")]
    public IActionResult Boom()
        => throw new InvalidOperationException("Absichtlicher Fehler");

    [HttpGet("missing")]
    public IActionResult Missing() => throw new KeyNotFoundException();
}
```

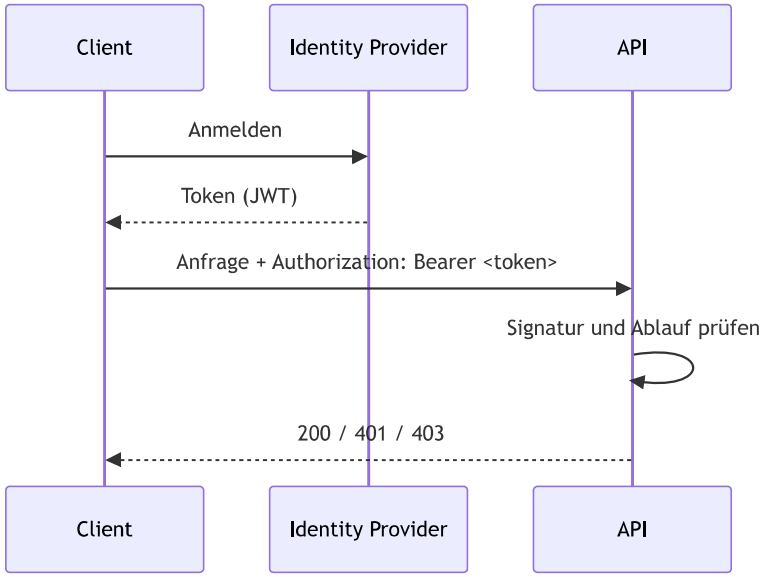
> Details nur ins Log. Tipp: `ctx.TraceIdentifier` in die Antwort – hilft dem Support.

Sicherheitsaspekte

Authentifizierung vs. Autorisierung

	Authentifizierung	Autorisierung
Frage	Wer bist du?	Was darfst du?
Bild	Ausweis am Eingang	Zutritt zum Serverraum
Scheitert →	`401 Unauthorized`	`403 Forbidden`
Middleware	`UseAuthentication()`	`UseAuthorization()`

Token-basierte Authentifizierung



Identity Provider: Microsoft Entra ID, Keycloak, Auth0 … · Die API sieht nie ein Passwort.

Was ist ein JWT?

```
eyJhbGciOiJIUzI1NiJ9 . eyJuYW1lIjoiYW5uYSIsInJvbGUiOiJBZG1pbiJ9 . 4pC2…
       Header                        Payload (Claims)                Signatur
```

Teil	Inhalt
Header	Algorithmus, Typ
Payload	**Claims**: Name, Rollen, Ablaufzeit, Aussteller
Signatur	schützt vor Veränderung

> Die Payload ist nur kodiert, **nicht verschlüsselt** – keine Geheimnisse ins Token.

Zugriffssteuerung mit [Authorize]

```
[Authorize]                                 // ganzer Controller geschützt
public class SecureController : ControllerBase
{
    [HttpGet, AllowAnonymous]               // Ausnahme: öffentlich
    public string Get() ⇒ "Für alle";

    [HttpPost]                              // nur angemeldet
    public string Post() ⇒ "Angemeldet";

    [HttpDelete, Authorize(Roles = "Admin")]
    public string Delete() ⇒ "Nur Admins";
}
```

Muster: standardmäßig geschützt, gezielt öffnen.

Weitere Sicherheitsgrundlagen

- **HTTPS** immer – in der Vorlage aktiv
- **Eingaben validieren** – Tag 2
- **Keine Details in Fehlermeldungen** – gerade umgesetzt
- **Geheimnisse nicht in Git** – User Secrets, Tresor
- **Abhängigkeiten prüfen**: `dotnet list package --vulnerable`
- Checkliste: **OWASP API Security Top 10**

Benutzer und Rollen mit JWT

JWT-Authentifizierung einrichten

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

```
builder.Services.AddAuthentication().AddJwtBearer();
builder.Services.AddAuthorization();
// ...
app.UseAuthentication();   // zuerst: Wer bist du?
app.UseAuthorization();    // dann: Was darfst du?
```

Test-Tokens mit dotnet user-jwts

```
dotnet user-jwts create                    # normaler Benutzer
dotnet user-jwts create --role Admin       # mit Rolle
dotnet user-jwts create --name anna        # mit Namen
dotnet user-jwts list                      # alle Tokens
dotnet user-jwts print <id>                # Token anzeigen
```

- Erzeugt lokal gültige Tokens
- Trägt die Konfiguration in `appsettings.Development.json` ein

Rollen und Policies

Rolle direkt

```
[Authorize(Roles = "Admin")]
public IActionResult Delete() ...
```

Policy

```
builder.Services.AddAuthorization(o ⇒
    o.AddPolicy("AdminOnly",
        p ⇒ p.RequireRole("Admin")));

[Authorize(Policy = "AdminOnly")]
public IActionResult Delete() ...
```

	Rolle	Policy
einfach	ja	ja
mehrere Bedingungen	nein	ja
Claims prüfen	nein	ja
zentral änderbar	nein	ja

Rate Limiting

```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    o.AddFixedWindowLimiter("fixed", l =>
    {
        l.PermitLimit = 5;
        l.Window = TimeSpan.FromSeconds(10);
    });
});
app.UseRateLimiter();

[HttpGet, EnableRateLimiting("fixed")]
```

Algorithmen: Fixed Window · Sliding Window · Token Bucket · Concurrency

CORS

```
builder.Services.AddCors(o => o.AddPolicy("Frontend", p => p
    .WithOrigins("http://localhost:4200")
    .AllowAnyHeader()
    .AllowAnyMethod()));
// ...
app.UseCors("Frontend");
```

- Browser blockieren JavaScript-Aufrufe an fremde Domains
- CORS erlaubt bestimmte Herkünfte
- **Kein Schutz der API –** curl und Postman ignorieren CORS

Aufrufer einschränken

IP-Allowlist als Middleware

```
using System.Net;

var intern = IPNetwork.Parse("10.0.0.0/24");

app.Use(async (ctx, next) ⇒
{
    var ip = ctx.Connection.RemoteIpAddress;
    if (ip?.IsIPv4MappedToIPv6 == true) ip = ip.MapToIPv4();
    if (ip is null || !(IPAddress.IsLoopback(ip) || intern.Contains(ip)))
    {
        ctx.Response.StatusCode = StatusCodes.Status403Forbidden;
        return;
    }
    await next();
});
```

Besser vor der API

- Firewall, privates Netz, Reverse Proxy
- Kestrel nur intern binden: `"Urls": "http://10.0.0.1:5000"`

Achtung

- `AllowedHosts` prüft nur den **Host-Header**, nicht den Aufrufer
- Hinter Proxy ist die IP die des Proxys → `UseForwardedHeaders`
- IP ist keine Identität → zusätzlich **Authentifizierung**

Übung 15 – Benutzer, Rollen und Einschränkungen

20 Minuten

`SecureController`:

- `GET` öffentlich
- `POST` nur für angemeldete Benutzer
- `DELETE` nur für die Rolle „Admin“

Testen: ohne Token · normales Token · Admin-Token

Bonus: Rate Limiting auf `GET`

Lösung Übung 15 (1/2)

```
builder.Services.AddAuthentication().AddJwtBearer();
builder.Services.AddAuthorization(o ⇒
    o.AddPolicy("AdminOnly", p ⇒ p.RequireRole("Admin")));
builder.Services.AddRateLimiter(o ⇒
{
    o.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    o.AddFixedWindowLimiter("fixed", l ⇒
    { l.PermitLimit = 5; l.Window = TimeSpan.FromSeconds(10); });
});
// ...
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();
```

Lösung Übung 15 (2/2)

```
[ApiController]
[Route("api/[controller]")]
[Authorize]
public class SecureController : ControllerBase
{
    [HttpGet, AllowAnonymous, EnableRateLimiting("fixed")]
    public string Get() => "Für alle sichtbar";

    [HttpGet("me")]
    public object Me() => new { Name = User.Identity?.Name,
                                IstAdmin = User.IsInRole("Admin") };
    [HttpPost]
    public string Post() => "Nur für angemeldete Benutzer";

    [HttpDelete, Authorize(Policy = "AdminOnly")]
    public string Delete() => "Nur für Admins";
}
```

Erwartete Ergebnisse Übung 15

Anfrage	Ohne Token	Normales Token	Admin-Token
`GET`	200	200	200
`POST`	**401**	200	200
`DELETE`	**401**	**403**	200
`GET` > 5× in 10 s	**429**	**429**	**429**

```
POST {{host}}/api/secure
Authorization: Bearer {{token}}
```

Caching

Warum Caching?

- Daten ändern sich selten, werden aber oft gelesen
- Weniger Datenbanklast, schnellere Antworten
- Preis: Daten können **veraltet** sein

> Die zentrale Frage: Wie alt darf es sein, und wann wird der Cache geleert?

Caching-Varianten in ASP.NET

Variante	Wo	Steuerung
Output Caching	Server, ganze Antwort	Attribut
IMemoryCache	Server, beliebige Werte	im Code
IDistributedCache	Redis, mehrere Server	im Code
Response Caching	Browser, Proxy	HTTP-Header

Output Caching

```
builder.Services.AddOutputCache();
//  ...
app.UseOutputCache();

[HttpGet, OutputCache(Duration = 30, Tags = new[] { "slow" })]
public async Task<string> Get() {  ... }

// Cache gezielt leeren
await store.EvictByTagAsync("slow", ct);
```

> Standard: nur `GET` / `HEAD`, Status 200, **anonyme** Anfragen.

IMemoryCache

```
builder.Services.AddMemoryCache();

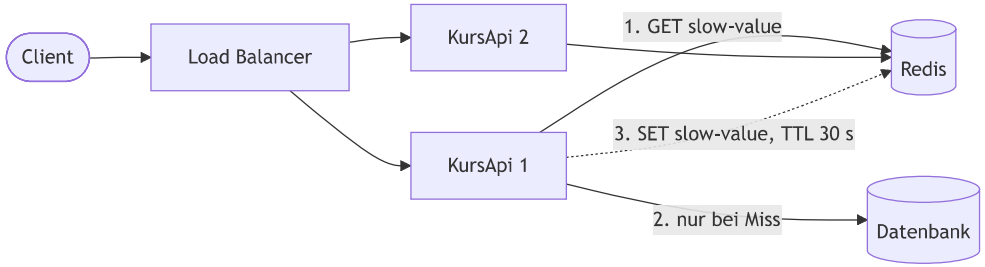
[HttpGet("memory")]
public async Task<string?> GetMemory([FromServices] IMemoryCache cache)
{
    return await cache.GetOrCreateAsync("slow-value", async entry ⇒
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromSeconds(30);
        await Task.Delay(3000);
        return $"Berechnet um {DateTime.Now:T}";
    });
}
```

Was ist Redis?

- **In-Memory-Datenbank**: Schlüssel-Wert-Speicher, Daten liegen im **RAM**
- Eigener Server-Prozess, Zugriff übers Netz (Port `6379`)
- Antwortzeiten im Bereich von **unter einer Millisekunde**
- Jeder Schlüssel kann eine **Ablaufzeit** (TTL) haben – ideal für Caches
- Optional auf Platte gesichert, übersteht dann auch einen Neustart

Einsatz	Beispiel
Cache	Ergebnis einer teuren Abfrage
Sessions	Anmeldedaten über mehrere Server
Zähler, Rate Limits	Anfragen pro Benutzer
Nachrichten	Pub/Sub zwischen Diensten

Wie funktioniert Redis als Cache?



- **Cache-Aside**: zuerst Redis fragen, nur bei einem **Miss** rechnen und speichern
- Alle Instanzen teilen **einen** Cache – `IMemoryCache` hätte pro Server einen eigenen Stand
- Redis kennt nur Strings und Bytes → Objekte selbst als JSON serialisieren

Verteilter Cache mit Redis

```
docker run -d -p 6379:6379 redis
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

```
builder.Services.AddStackExchangeRedisCache(o ⇒ o.Configuration = "localhost:6379");

[HttpGet("redis")]
public async Task<string> GetRedis([FromServices] IDistributedCache cache)
{
    var value = await cache.GetStringAsync("slow-value");
    if (value is null)
    {
        await Task.Delay(3000);
        value = $"Berechnet um {DateTime.Now:T}";
        await cache.SetStringAsync("slow-value", value, new DistributedCacheEntryOptions
            { AbsoluteExpirationRelativeToNow = TimeSpan.FromSeconds(30) });
    }
    return value;
}
```

Nur Strings/Bytes · alle Server sehen denselben Cache · überlebt Neustarts der API

Übung 16 – Caching

15 Minuten

1. Langsamer Endpoint: `await Task.Delay(3000)`, Zeitstempel zurückgeben
2. Mit **Output Caching** beschleunigen
3. Antwortzeiten vergleichen

Bonus: dieselbe Lösung mit `IMemoryCache` oder Redis · Cache über einen Tag gezielt leeren

Kein Effekt? Authorization-Header entfernen – nur anonyme Anfragen werden gecacht.

Lösung Übung 16 (1/2)

```
[ApiController]
[Route("api/[controller]")]
public class SlowController : ControllerBase
{
    [HttpGet, OutputCache(Duration = 30, Tags = new[] { "slow" })]
    public async Task<string> Get()
    {
        await Task.Delay(3000);
        return $"Berechnet um {DateTime.Now:T}";
    }
```

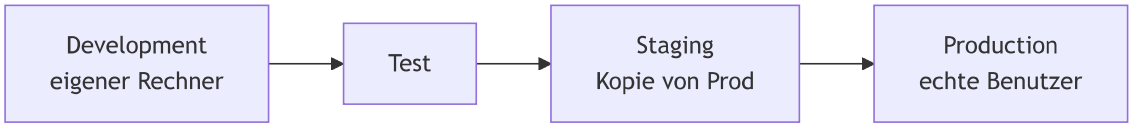
Lösung Übung 16 (2/2)

```
    [HttpPost("clear")]
    public async Task<IActionResult> Clear(
        [FromServices] IOutputCacheStore store, CancellationToken ct)
    {
        await store.EvictByTagAsync("slow", ct);
        return NoContent();
    }
}
```

Die `IMemoryCache`-Variante steht auf der Folie „IMemoryCache“.

Deployment und Hosting

Umgebungen



- Derselbe Build durchläuft alle Umgebungen
- Es ändert sich nur die **Konfiguration**

Die Umgebung im Code

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

```
dotnet run -- --environment Staging
ASPNETCORE_ENVIRONMENT=Staging dotnet KursApi.dll
```

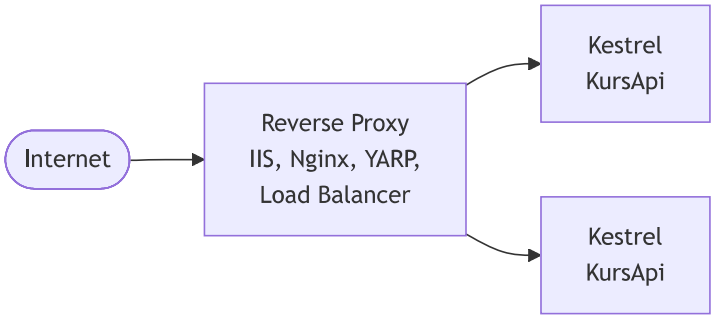
Methode	Prüft
`IsDevelopment()`	Development
`IsStaging()`	Staging
`IsProduction()`	Production

dotnet publish

```
dotnet publish -c Release -o ./publish
dotnet ./publish/KursApi.dll
```

- Optimierter Build mit allem, was die App braucht
- Server braucht nur die **ASP.NET Core Runtime**, kein SDK
- `--self-contained` : Runtime im Paket, kein .NET auf dem Server
- Ohne Launch-Profil: Standardport `http://localhost:5000`

Hosting-Modelle



- **Kestrel**: eingebauter Webserver, produktionstauglich
- Proxy übernimmt TLS, Lastverteilung · IIS braucht das Hosting Bundle

Übung 17 – Umgebungen und Publish

15 Minuten

1. `appsettings.Staging.json` mit anderem Begrüßungstext
2. App als Staging starten, `GET /api/config` aufrufen
3. `dotnet publish -c Release -o ./publish`
4. DLL direkt starten

Beobachten: Was passiert mit Swagger und mit dem JWT-Endpoint?

Lösung Übung 17

```
// appsettings.Staging.json
{ "Greeting": { "Text": "Hallo aus Staging!" } }
```

```
dotnet run -- --environment Staging
dotnet publish -c Release -o ./publish
dotnet ./publish/KursApi.dll --environment Staging
```

- Swagger aus – nur in Development aktiv
- JWT-Konfiguration fehlt – stand nur in `appsettings.Development.json`

Hosting in Docker

Warum Container?

„Auf meinem Rechner läuft es.“

Begriff	Bedeutung
Dockerfile	Bauanleitung
Image	fertige Vorlage mit App und Runtime
Container	laufende Instanz eines Images
Registry	Ablage für Images (Docker Hub, ACR)

Das Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app .
ENTRYPOINT ["dotnet", "KursApi.dll"]
```

Multi-Stage: SDK zum Bauen, kleine Runtime zum Laufen · Tags an die eigene .NET-Version anpassen

Bauen und starten

```
docker build -t kursapi .
docker run -p 8080:8080 -e ASPNETCORE_ENVIRONMENT=Staging kursapi

# Konfiguration per Umgebungsvariable
docker run -p 8080:8080 -e Greeting__Text="Hallo aus Docker" kursapi

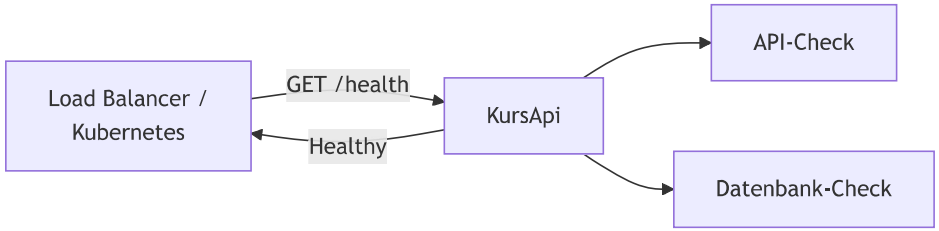
# Alternative ohne Dockerfile
dotnet publish /t:PublishContainer
```

> Ab .NET 8 lauscht das Image auf **Port 8080** und läuft nicht als root.

Überwachen der Anwendung

Säule	Frage	Werkzeug
Logs	Was ist passiert?	Serilog, Seq
Metriken	Wie viel, wie schnell?	OpenTelemetry
Traces	Welchen Weg nahm die Anfrage?	OpenTelemetry
Health Checks	Ist die App gesund?	`/health`

Health Checks



- Unhealthy → keine Anfragen mehr oder Neustart
- Kubernetes: **Liveness** (lebt der Prozess?) und **Readiness** (bereit für Anfragen?)

Health Checks einrichten

```
dotnet add package Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore
```

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<LibraryContext>()
    .AddCheck("api", () => HealthCheckResult.Healthy("API läuft"));
// ...
app.MapHealthChecks("/health");   // Healthy / Degraded / Unhealthy
```

Übung 18 – Health Checks

10 Minuten

1. Paket für den EF-Core-Check installieren
2. Endpoint `/health` mit Datenbank-Check
3. Aufrufen: `Healthy` ?

Bonus: eigener Check mit `AddCheck`

Lösung Übung 18

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<LibraryContext>()
    .AddCheck("api", () => HealthCheckResult.Healthy("API läuft"));

app.MapHealthChecks("/health");
```

> Test: `library.db` kurz umbenennen – der Check meldet `Unhealthy`.

Abschluss-Challenge

35 Minuten

Zitate-API betriebsbereit machen

Verwendet: alle Themen der vier Tage

> Handler, JWT und Health Check kommen aus den Übungen 14, 15 und 18. Neu ist das Zusammenspiel.

Abschluss-Challenge – Anforderungen

- `GET` öffentlich · `POST` angemeldet · `DELETE` nur Admins
- Unerwartete Fehler über den globalen Handler
- `GET /api/v2/quotes` 60 s gecacht, nach `POST` / `DELETE` geleert
- In Staging gilt eine andere maximale Anzahl
- `/health` meldet API und Datenbank
- Alle Fälle (ohne Token, mit Token, Admin) in der `.http`-Datei

Lösung Abschluss-Challenge (1/3)

```
[ApiController]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/quotes")]
[Authorize]
public class QuotesV2Controller : ControllerBase
{
    private readonly IQuoteService _service;
    private readonly IOutputCacheStore _cache;

    public QuotesV2Controller(IQuoteService service, IOutputCacheStore cache)
    {
        _service = service;
        _cache = cache;
    }
```

Lösung Abschluss-Challenge (2/3)

```
[HttpGet, AllowAnonymous, OutputCache(Duration = 60, Tags = new[] { "quotes" })]
public async Task<ActionResult<IEnumerable<Quote>>> GetAll()
    => Ok(await _service.GetAllAsync());

[HttpPost]
public async Task<ActionResult<Quote>> Create(Quote quote, CancellationToken ct)
{
    var created = await _service.AddAsync(quote);
    if (created is null) return Conflict("Maximale Anzahl Zitate erreicht.");
    await _cache.EvictByTagAsync("quotes", ct);
    return CreatedAtAction(nameof(GetById),
        new { id = created.Id, version = "2.0" }, created);
}
```

`GetById` wie in Challenge 3, zusätzlich mit `[AllowAnonymous]`

Lösung Abschluss-Challenge (3/3)

```
[HttpDelete("{id:int}"), Authorize(Policy = "AdminOnly")]
public async Task<IActionResult> Delete(int id, CancellationToken ct)
{
    if (!await _service.DeleteAsync(id)) return NotFound();
    await _cache.EvictByTagAsync("quotes", ct);
    return NoContent();
}
```

```
// appsettings.Staging.json
{ "Quotes": { "MaxQuotes": 3 } }
```

Middleware-Reihenfolge

```
app.UseExceptionHandler();      // 1. Fehler aller folgenden fangen

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthentication();        // 2. Wer bist du?
app.UseAuthorization();         // 3. Was darfst du?
app.UseRateLimiter();
app.UseOutputCache();
app.MapControllers();
app.MapHealthChecks("/health");
```

Datei-Upload

Zwei Wege für den Upload

	Multipart	Raw Binary
`Content-Type`	`multipart/form-data`	Typ der Datei, z. B. `image/png`
Body	Teile, getrennt durch Boundary	nur die Datei
Zusatzfelder	ja, auch mehrere Dateien	nein, nur über URL oder Header
In ASP.NET	`IFormFile`	`Request.Body`
Typisch	Browser-Formulare	Apps, Skripte, Dienste

> Bei beiden wird gestreamt: Die Datei gehört nicht komplett in den Arbeitsspeicher.

So sieht die Anfrage aus

Multipart

```
POST /api/files/multipart
Content-Type: multipart/form-data;
  boundary=KursGrenze

--KursGrenze
Content-Disposition: form-data;
  name="file"; filename="hallo.txt"
Content-Type: text/plain

Hallo
--KursGrenze--
```

Raw Binary

```
POST /api/files/raw
Content-Type: text/plain

Hallo
```

Der Name `file` im Formular muss zum Parameter der Aktion passen.

Die beiden Endpoints

```
[ApiController]
[Route("api/[controller]")]
[Authorize]
[RequestSizeLimit(5 * 1024 * 1024)]
public class FilesController : ControllerBase
{
    [HttpPost("multipart")]
    public async Task<IActionResult> UploadMultipart(IFormFile file, CancellationToken ct)
    {
        await using var source = file.OpenReadStream();
        return await SaveAsync(source, file.ContentType, ct);
    }

    [HttpPost("raw")]
    public Task<IActionResult> UploadRaw(CancellationToken ct)
        => SaveAsync(Request.Body, Request.ContentType, ct);
```

Die Datei speichern

```
private async Task<IActionResult> SaveAsync(Stream source, string? contentType, CancellationToken ct)
{
    var mediaType = contentType?.Split(';')[0].Trim();
    if (mediaType is null || !AllowedTypes.TryGetValue(mediaType, out var extension))
        return StatusCode(StatusCodes.Status415UnsupportedMediaType);

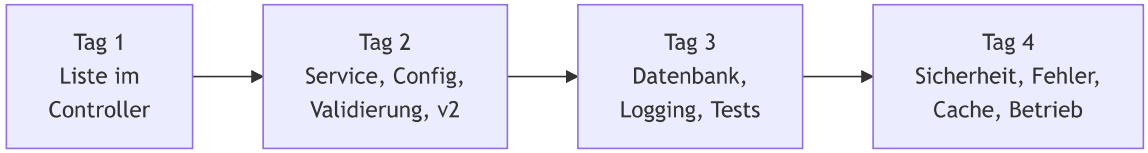
    Directory.CreateDirectory(_folder);
    var fileName = $"{Guid.NewGuid()}{extension}";
    var path = Path.Combine(_folder, fileName);
    try
    {
        await using var target = System.IO.File.Create(path);
        await source.CopyToAsync(target, ct);
        return Ok(new { fileName, size = target.Length });
    }
    catch (Exception ex)
    {
        System.IO.File.Delete(path);
        if (ex is BadHttpRequestException bad) return StatusCode(bad.StatusCode);
        throw;
    }
}
```

Uploads absichern

Regel	Warum
Dateiname vom Server	`../../evil.exe` darf nie im Pfad landen
Typen per Allowlist	alles andere → `415`
Größe begrenzen	Standard sind rund 30 MB, zu groß → `413`
Ablage außerhalb von `wwwroot`	Uploads dürfen nicht direkt abrufbar sein
Nur mit `[Authorize]`	sonst ist die Platte für alle offen

> `Content-Type` ist nur eine Behauptung des Clients. Wer sicher sein will, prüft den Dateiinhalt.

Gesamtrückblick



> Der Weg, den echte APIs auch gehen.

Wie geht es weiter?

- **Minimal APIs** – die kompakte Alternative zu Controllern
- **Integrationstests** mit `WebApplicationFactory`
- **OpenTelemetry** für Metriken und Traces
- **Redis** als verteilter Cache
- Echter **Identity Provider**: Entra ID, Keycloak
- **CI/CD** mit GitHub Actions oder Azure DevOps

Dokumentation: learn.microsoft.com/aspnet/core

Offene Fragen

Was ist noch offen?

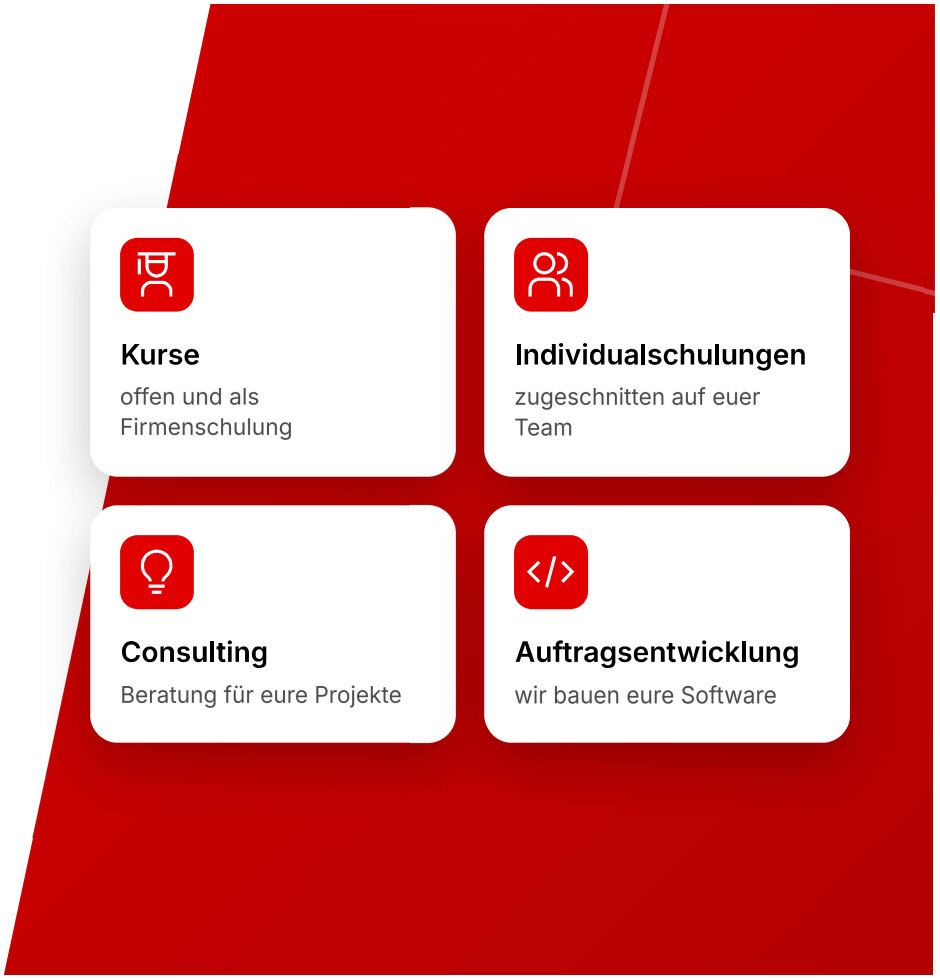
Mühlehner & Tavolato GmbH

SCHULUNG · BERATUNG · ENTWICKLUNG

AF

Alexander Fischer

af@mit.at



Danke!

Viel Erfolg mit euren eigenen APIs.

Alle Unterlagen: github.com/al3xfischer/aspnet_unterlagen